Revisiting Sequential Consistency

Memory Consistency Models, Their Cost, and Whether Speculation Has Made Strong Ordering Affordable

Term Paper — Version 0.1

ELL305: Computer Architecture

Indian Institute of Technology Delhi

Author *[Your Name]*
Entry No. *[Your Entry Number]*
Instructor *[Instructor]*

Date August 22, 2026

Prescribed text: S. R. Sarangi, *Basic Computer Architecture*, v3.09.
Primary coverage: Chapter 12, §12.4 (MIMD Multiprocessors).

Abstract

A shared–memory multiprocessor must answer a question that a uniprocessor never has to face: when one core writes to memory, when — and in what order relative to its other writes — must the other cores see that write? The answer a machine gives is called its *memory consistency model*. It is not an implementation detail but an architectural contract, as binding as the instruction set itself, and it is the reason a multithreaded C program that is provably correct on paper can fail on real silicon.

The obvious contract is *sequential consistency* (SC), formalised by Lamport in 1979: the machine must behave as though the memory operations of all cores were interleaved into a single total order that respects each core's program order. SC matches programmer intuition exactly. By the early 1990s, however, the architecture community had rejected it for hardware implementation on performance grounds, and every commercially significant instruction set since — SPARC, Alpha, POWER, ARM, and RISC-V — has published a weaker contract, exposing reordering to software and supplying fence instructions with which programmers are expected to suppress it selectively.

That decision was reached on machines with in-order, non-speculative pipelines, small bus-snooped caches, and core counts in the single digits. Every one of those premises has since been invalidated. A modern out-of-order core already contains both mechanisms that a strong model needs: a coherence protocol that announces remote writes, and a squash-and-restart path built for branch misprediction recovery. A line of work beginning with Gniady et al. has argued that, given these, SC can be enforced speculatively at a small fraction of its assumed cost. And yet no shipping processor implements it.

This paper examines that gap. It develops a single formal framework in which SC, TSO, PSO, weak ordering and release consistency appear as successively weaker constraints on one set of ordering relations; it traces the historical argument from 1979 to the ratification of RVWMO; it establishes, by direct measurement on x86 and ARM hardware, which relaxations are observable in practice as opposed to merely permitted; and it quantifies, in a configurable multicore simulator, the cost of enforcing each model with and without speculative load execution. The governing question is whether the performance objection to sequential consistency still holds on a machine that already speculates — and, if it does not, what else explains the industry's continued preference for weak ordering.

Note on this version. This is draft v0.1. It comprises the formal, historical and methodological portions of the paper (Chapters 1–6). The experimental and simulation results announced in Chapter 6 are the subject of versions v0.2 and v0.3; Chapter 6 states the design of those experiments in sufficient detail to be evaluated in advance of the data.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 A Program That Should Not Fail

Consider the fragment of code in Listing 1.1. Two threads run concurrently on two cores of an ordinary multicore processor. Two shared variables, `x` and `y`, are both initialised to zero. Each thread writes 1 to one of them and then reads the other.

```
1 /* initially: x == 0, y == 0 */
2
3 /* Thread 0 */                /* Thread 1 */
4 x = 1;                        y = 1;
5 r0 = y;                       r1 = x;
```

Listing 1.1: The store-buffer idiom. Both threads write one shared variable and then read the other.

Ask the following question: on termination, can both `r0` and `r1` hold the value zero?

The argument that they cannot is short and appears airtight. The two stores are distinct events; one of them must occur before the other. Suppose without loss of generality that `x = 1` occurs first. Thread 1 reads `x` only after it has written `y`, and it wrote `y` after `x = 1` occurred. Hence Thread 1's read of `x` follows the store to `x`, and must return 1, so `r1` = 1. By symmetry the same holds if `y = 1` occurs first. The outcome `r0` = `r1` = 0 therefore requires each read to have preceded a store that preceded it, which is a contradiction.

This argument is valid. It is also, on essentially every processor sold today, empirically false. Compiled without synchronisation and run in a loop on a commodity x86 machine, the outcome `r0` = `r1` = 0 occurs. On an ARM machine it occurs more often, and so do several further outcomes that x86 never produces.

The argument fails not because any step of it is wrong, but because of an unstated premise: that a store, once executed by a core, is immediately visible to every other core. That premise is false, and it is false by deliberate design. It is discarded in exchange for performance, and the mechanism that discards it — the *store buffer* — is present in every high-performance core built in the last thirty years.

The consequence is not confined to artificial examples. Listing 1.1 is the core of Dekker's algorithm, the first published solution to mutual exclusion. Executed on hardware that permits the outcome above, Dekker's algorithm admits two threads into the critical section simultaneously. A correctness proof that has stood since 1965 does not transfer to the machine.

1.2 Coherence Is Not Enough

A natural first response is that the machine in question must have a defective cache. It does not. The behaviour survives a perfectly correct implementation of cache coherence.

Sarangi draws the distinction in §12.4.2 and §12.4.3 of the prescribed text [15], and it is worth stating sharply because much of the confusion surrounding this topic originates in conflating the two notions.

Coherence is a guarantee about a single memory location. It requires that all cores observe the writes to any one address in a single agreed order, and that a read of that address returns the most recent value in that order. Protocols such as MSI, MESI and MOESI, treated in §12.4.6 of the text and in Chapter 3 of this paper, exist to provide exactly this. They are well understood and correctly implemented in practice.

Consistency is a guarantee about the relative order of accesses to *different* locations. Listing 1.1 involves two addresses. Coherence constrains the writes to `x` among themselves and the writes to `y` among themselves; it says nothing whatever about how Thread 0's store to `x` is ordered with respect to Thread 0's load of `y`. The anomalous outcome requires no violation of coherence and does not produce one.

Definition 1.1 (Memory consistency model)**.** A *memory consistency model* is the specification, forming part of a machine's architecture, of which orderings among memory operations issued by a single core are guaranteed to be preserved as seen by other cores, together with the instructions provided to the programmer for enforcing orderings that are not guaranteed by default.

The model is a contract. Like the instruction set, it is an interface: hardware promises no less, software may assume no more. Unlike the instruction set, it is frequently absent from a programmer's mental model of the machine entirely, which is precisely why the failures it permits are so difficult to diagnose.

1.3 The Design Tension

Figure 1.1 states the situation that this paper investigates. Two objectives are in conflict.

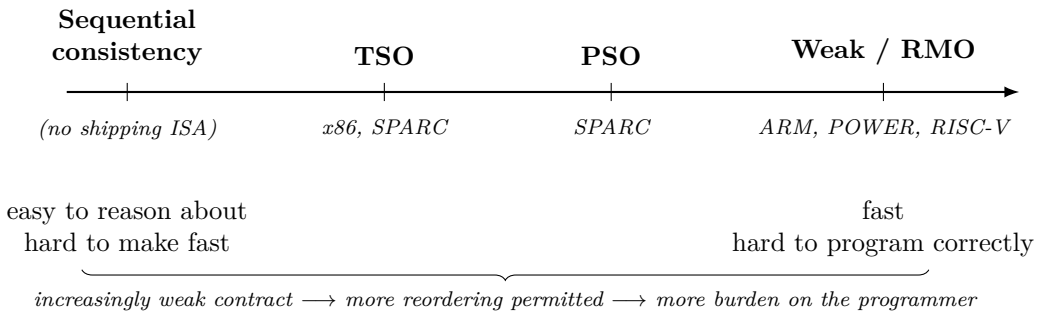

Figure 1.1: The design space of hardware memory consistency models. Every commercially deployed instruction set sits to the right of sequential consistency.

At the left-hand end of the spectrum the hardware guarantees everything, and the reasoning of §1.1 is sound. The programmer needs no special instructions and no special knowledge. The cost is that the core must, in the naive implementation, stall on every store until that store is globally visible — which is to say, it must abandon the store buffer, and with it a substantial fraction of its performance.

At the right-hand end the hardware guarantees almost nothing and runs at full speed. The programmer is handed *fence* instructions and made responsible for placing

them wherever an ordering is genuinely required. Empirically, programmers place them incorrectly. The resulting defects are probabilistic, appear only under particular thread placements and core counts, and typically vanish under a debugger. Two decades of experience with the Java Memory Model [11], the C++ memory model [5] and the Linux kernel's own memory model attest to the difficulty.

The industry chose the right-hand end. The justification was performance, and it was articulated in an era whose assumptions no longer hold.

1.4 Why the Question Is Open

Three developments since 1990 bear directly on the original trade-off.

Speculation is now ubiquitous. A modern out-of-order core executes instructions ahead of their architectural order and discards the results when a prediction turns out to have been wrong. It therefore already possesses a squash-and-restart mechanism, built for branch misprediction recovery, and it already receives coherence invalidation messages announcing remote writes. Together these are sufficient to enforce sequential consistency *optimistically*: execute the load early, and roll back only in the rare case where a remote write proves that doing so was observable. Gniady, Falsafi and Vijaykumar posed this directly [9], and later work extended it to speculative store retirement and to chunk-based execution [6, 4].

Compilers matter as much as hardware. Hardware sequential consistency is worthless if the compiler reorders shared accesses first. For a long time it was assumed that constraining the compiler would be prohibitively expensive. Marino et al. measured it and found the cost modest [12].

Core counts have grown by an order of magnitude. The measurements that settled the original debate were taken on machines with between two and sixteen processors. Speculation-based SC depends on conflicts being rare; whether that premise survives at 64 or 128 cores is not established by any of the classical results.

Against all of this stands one stubborn observation: *no shipping processor implements sequential consistency.* If the performance objection has been answered, something else is keeping the industry where it is — energy, verification cost, the irreversibility of an architectural commitment, or simple inertia.

1.5 Problem Statement

> Sequential consistency was rejected as a hardware memory model on performance grounds in the late 1980s, on machines that did not speculate. This paper asks whether that verdict still holds. It (i) formalises the design space between SC and the weak models actually deployed within a single ordering framework; (ii) establishes by direct measurement on x86 and ARM hardware which relaxations are observable in practice, as distinct from those merely permitted by the architecture; and (iii) quantifies, in a configurable multicore simulator, the execution-time cost of enforcing each model both with and without speculative load execution, across core counts beyond those examined in the classical literature.

The problem decomposes into four questions.

Q1 (Formal). Can SC, TSO, PSO, weak ordering and release consistency be expressed as successively weaker constraints on a single set of relations over memory events, and does the resulting hierarchy admit a proof of strict containment? Chapter 4.

Q2 (Empirical). For a catalogue of litmus tests, which outcomes are permitted by each architecture's specification, and which are actually produced by hardware? Where the two differ, what accounts for the difference? Chapters 6–7.

Q3 (Quantitative). Holding the microarchitecture fixed and varying only the memory model, what is the execution-time cost of SC relative to TSO and weak ordering, with and without speculative load execution, as a function of core count? Chapters 8–9.

Q4 (Interpretive). If the measured cost of speculative SC is small, what accounts for its absence from shipping hardware? Chapter 11.

1.6 Contributions

This paper claims the following.

1. A unified presentation of five memory consistency models within the execution-witness formalism of Sarangi [14], with the containment hierarchy proved rather than asserted.
2. A litmus-test suite, executed on both x86 and ARM hardware, reporting observation frequencies rather than the usual binary permitted/forbidden classification — together with a measured cost, in cycles, for the fence instructions that suppress each relaxation.
3. A multicore simulator in which the memory consistency model is an interchangeable component, permitting a controlled comparison in which the model is the only variable.
4. A verification chapter in which the simulator's exhaustively enumerated outcomes are checked against the axiomatic definitions of Chapter 4, so that the performance results rest on an implementation demonstrated to implement the model it claims.
5. A scaling study extending the speculative-SC question to core counts beyond those examined in the 1999–2011 literature.

1.7 Relation to the Prescribed Text

The paper is anchored in Chapter 12 of Sarangi [15], and specifically in §12.4. Table 1.1 maps each chapter of this paper onto the sections of the text it extends.

Table 1.1: Coverage of the prescribed text.

This paper	Text	Relationship
Ch. 2 Background	§11.2, §11.3, §10.11.4, §12.2	Caches, the memory system, out-of-order pipelines, shared memory versus message passing.
Ch. 3 Coherence	§12.4.2, §12.4.5, §12.4.6, §12.6	Extends the text's treatment of coherent private caches to full MESI and MOESI transition tables and to directory protocols.
Ch. 4 Models	§12.4.3, §12.4.7	The core section of the text. Extended to a formal treatment of four further models and a containment proof.
Ch. 5 History	§12.7.2	Develops the further-reading pointers into a chronology.
Ch. 6 Methodology	§12.4.3, §10.9	Applies the text's performance-metric framework to litmus testing.
Ch. 8–9 Simulation	§10.9, §12.4.7, §12.6	Performance equation, consistency implementation, interconnect modelling.
Ch. 11 Discussion	§12.2.3, App. A	Amdahl's law; the Cortex-A8, Cortex-A15 and Sandy Bridge case studies.

Where the treatment exceeds the scope of the prescribed text — notably the axiomatic formalism, the relaxed models beyond TSO, and the speculative implementation techniques — the primary sources are Chapter 9 of Sarangi's companion volume [14] and the memory-consistency primer of Nagarajan et al. [13].

1.8 Organisation of the Paper

Chapter 2 develops the microarchitectural background: why memory is slow, what caches and store buffers do about it, and how out-of-order execution compounds the reordering they introduce. Chapter 3 treats cache coherence and establishes precisely what it does and does not guarantee. Chapter 4 is the formal core: it defines the ordering relations and expresses each consistency model as a constraint over them. Chapter 5 traces the historical argument from Lamport's 1979 paper to the ratification of RVWMO. Chapter 6 sets out the litmus-test methodology and experimental design for the results reported in later versions of this paper.

Scope of version 0.1. The present version comprises Chapters 1–6. Chapters 7–12, containing the hardware measurements, the simulator and its results, the verification study and the discussion, are the subject of versions v0.2 and v0.3.

Chapter 2

Background: Why Memory Operations Are Reordered

This chapter establishes the microarchitectural facts on which the rest of the paper depends. None of the mechanisms described here was introduced in order to weaken the memory model; each was introduced to solve a performance problem, and the weakening of ordering guarantees is a side effect. That the side effect is unavoidable — that the speed and the reordering are two descriptions of the same property — is the central observation of the chapter.

2.1 The Memory Wall

A processor core executes arithmetic at a rate of one or more operations per clock cycle. Access to main memory takes on the order of 200 cycles. Sarangi develops this asymmetry in §11.1.1 of the prescribed text [15]; the consequence is that a core that waited for every memory access would spend the overwhelming majority of its cycles idle.

Two structures address this, and they address different halves of the problem. The *cache* shortens read latency by keeping copies of recently accessed lines close to the core. The *store buffer* eliminates write latency by allowing the core to retire a store without waiting for it to reach memory at all.

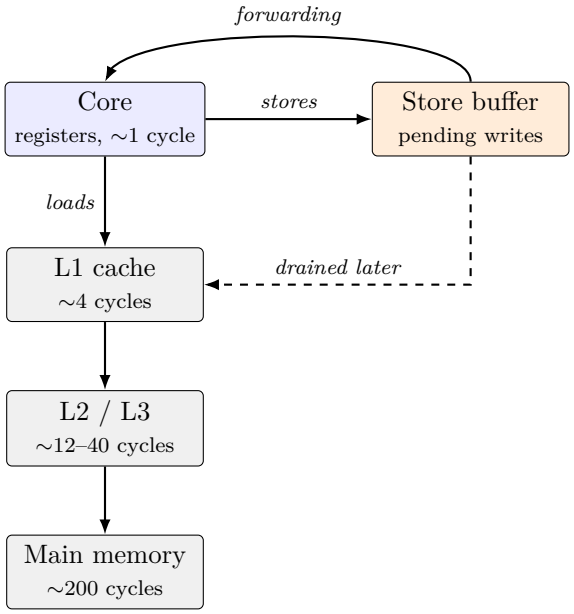

Figure 2.1: The memory path of a single core. Loads traverse the cache hierarchy; stores are placed in the store buffer and drained asynchronously. The forwarding path returns a pending store's value to a subsequent load from the same core.

2.2 Caches

Caches exploit temporal and spatial locality (§11.1.3–§11.1.6 of the text) to serve most accesses without a memory transaction. For the purposes of this paper three properties matter.

Caches are private. In a multicore processor, the L1 and typically the L2 cache belong to a single core. A line may therefore be resident in several caches simultaneously, and the copies may disagree unless something is done about it. That something is coherence, and it is the subject of Chapter 3.

Cache misses are non-blocking. A modern L1 permits multiple outstanding misses, tracked in miss-status holding registers. Consequently two loads issued in program order may complete in either order, depending on which one hits.

Cache line granularity. Coherence operates on lines, typically 64 bytes, not on individual words. Two variables that are logically unrelated but share a line will interact through the coherence protocol. This phenomenon, false sharing, will matter in Chapter 6 when litmus test variables are laid out.

2.3 The Store Buffer

The store buffer is the mechanism most directly responsible for the behaviour described in §1.1, and it deserves careful statement.

When a core executes a store, the value to be written is already in hand. Unlike a load, the store produces no result that subsequent instructions require. There is therefore no data dependence forcing the core to wait for the store to complete. The store buffer exploits this: the store is placed in a small FIFO queue — typically 40 to 60 entries —

and the core proceeds immediately. A separate mechanism drains the queue into the cache in the background.

The cost avoided is larger than it first appears. Writing to a cache line requires that the line be held in a state permitting modification. If it is not, the core must first issue a coherence request for exclusive ownership and wait for every other core holding a copy to relinquish it. On a large multicore that round-trip may take hundreds of cycles. Without a store buffer, the core stalls for all of it.

2.3.1 Store-to-Load Forwarding

If a core stores to an address and subsequently loads from the same address, the correct value may still be sitting in the store buffer rather than in the cache. The load therefore checks the store buffer before the cache, and if a matching pending store is found the value is returned directly. This is *store-to-load forwarding*.

Forwarding is why single-threaded programs are unaffected by any of this. A core always observes its own writes in program order, whatever the rest of the system sees. The uniprocessor programming model is preserved exactly; only the inter-core view is altered.

2.3.2 The Unavoidable Consequence

We can now state the point precisely.

> The store buffer is fast *because* a store held in it is not yet visible to other cores. The invisibility is not an implementation defect that better engineering could remove; it is the same fact as the performance benefit, viewed from the other side. Any mechanism that made pending stores immediately visible would, by definition, require the core to wait for them, which is the stall the store buffer exists to avoid.

Figure 2.2 traces Listing 1.1 through this structure and shows how the outcome `r0` = `r1` = 0 arises without any component behaving incorrectly.

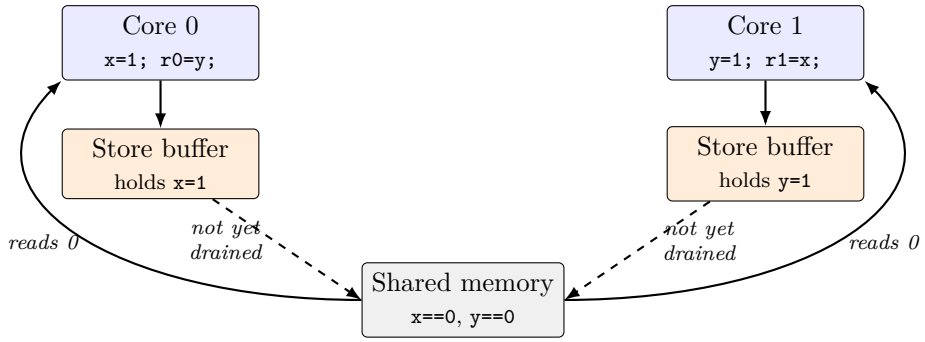

Figure 2.2: Execution of Listing 1.1. Each store is buffered locally while the corresponding load is served from shared memory, which still holds the initial values. Both loads return zero. No component has malfunctioned.

2.4 Out-of-Order Execution

Store buffering relaxes the ordering of a store with respect to *later* operations. Out-of-order execution, treated in §10.11.4 of the prescribed text, relaxes considerably more.

An out-of-order core fetches instructions in program order, issues them for execution as soon as their operands are available regardless of order, and retires them in program order so that exceptions remain precise. The load-store queue tracks memory operations, resolves addresses as they become known, and permits loads to be issued speculatively ahead of older stores whose addresses are not yet computed.

Three consequences follow for the memory model:

1. **Loads may execute out of order with respect to one another**, since a load that hits in the cache completes before an older load that misses.
2. **Loads may execute ahead of older stores.** This is a deliberate optimisation; a store's address may not be known for many cycles, and blocking all subsequent loads until it is would be very costly.
3. **A squash mechanism already exists.** When a branch is mispredicted, the core discards all speculative state after the branch and restarts. This machinery is general and, as Chapter 4 and the discussion of speculative SC will show, can be repurposed to recover from ordering violations.

The third point is the one on which this paper's argument turns. The expensive component of an optimistic implementation of sequential consistency — the ability to undo work — is not an addition to a modern core. It is already there.

2.5 The Compiler

Hardware is not the only source of reordering. An optimising compiler reasons about one thread at a time and freely reorders, coalesces, hoists and eliminates memory accesses that no single-threaded observer could distinguish. Register allocation alone can remove a shared-variable read entirely, converting a spin loop into an infinite loop.

The relevance to this paper is twofold. First, litmus tests must be written so that the compiler does not optimise away the very accesses being measured; the techniques for this are set out in Chapter 6. Second, any claim about the cost of sequential consistency must account for the compiler, since hardware SC without compiler SC provides no guarantee to the programmer at all. Marino et al. address precisely this [12], and Chapter 11 returns to it.

2.6 Summary

Table 2.1: Mechanisms that reorder memory operations, and their motivation.

Mechanism	Introduced to	Reordering permitted
Store buffer	hide store latency	store → later load
Non-blocking cache	overlap misses	load → load
Out-of-order issue	extract parallelism	load → load, load → store
Write coalescing	reduce bus traffic	store → store
Compiler optimisation	reduce instruction count	all of the above

Every entry in Table 2.1 is a performance mechanism whose ordering consequence is incidental. The architectural question — which of these reorderings a machine will admit to performing, and which it will hide from the programmer — is the subject of Chapter 4. First, however, Chapter 3 must dispose of the mechanism that is commonly, and mistakenly, expected to solve the problem.

Chapter 3

Cache Coherence, and What It Does Not Solve

Chapter 2 established that private caches admit multiple copies of a line. Coherence is the discipline that keeps those copies in agreement. This chapter develops it far enough to make two points precise: what coherence guarantees, and — more importantly for this paper — what it leaves entirely unconstrained.

3.1 The Coherence Invariants

Following the formulation of Nagarajan et al. [13], a memory system is coherent if it maintains two invariants.

Single-writer, multiple-reader (SWMR). For any memory location and any point in logical time, either exactly one core may write the location and no core may read it, or any number of cores may read it and none may write.

Data value. The value of a location at the start of an epoch is the value written at the end of the last read–write epoch for that location.

The SWMR invariant is the operative one. It partitions the lifetime of each line into epochs, alternating between a single exclusive writer and a set of concurrent readers, and the protocol's task is to enforce that partition.

An equivalent characterisation, and the one used in Sarangi's companion volume [14], is that coherence is *per-location sequential consistency* (PLSC): for each address considered in isolation, the machine behaves as though all accesses to that address occurred in a single total order consistent with each core's program order. Stated this way, the relationship to consistency becomes clear and is developed in §3.4.

3.2 Protocol Families

3.2.1 Write-Invalidate versus Write-Update

Two families satisfy SWMR. A *write-invalidate* protocol requires a core intending to write to first invalidate all other copies. A *write-update* protocol instead broadcasts the new value to all holders. The prescribed text treats both (§12.4.6); invalidate protocols dominate in practice, because update protocols consume interconnect bandwidth proportional to the number of sharers regardless of whether those sharers will ever read the line again. All protocols considered in this paper are invalidate-based.

3.2.2 MSI

The minimal invalidate protocol assigns each cached line one of three states.

M (Modified). This cache holds the only valid copy, and it differs from memory. The core may read or write freely.

S (Shared). This cache holds a valid, clean copy; others may too. Reads are permitted, writes are not.

I (Invalid). No usable copy is present.

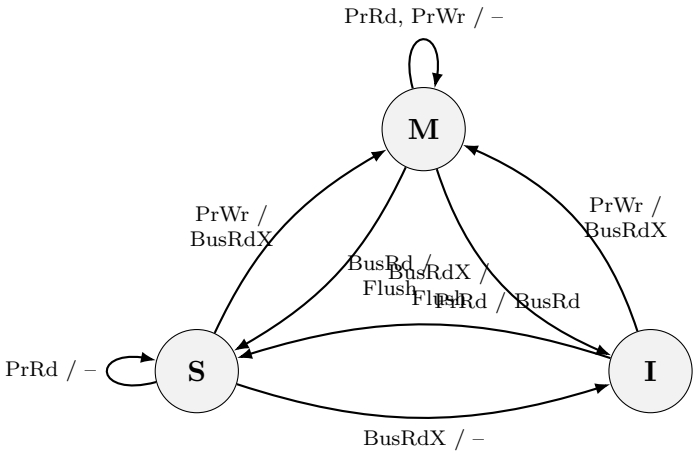

Figure 3.1: The MSI protocol. Transitions are labelled *event / action*, where `Pr` denotes a request from the local processor and `Bus` a request observed on the interconnect.

3.2.3 MESI and MOESI

MSI is correct but wasteful. A core that reads a line no other core holds, and then writes it, must issue two bus transactions: a `BusRd` to enter S, and a `BusRdX` to reach M. Since private, unshared data is the common case, this is a frequent and avoidable cost.

MESI adds an **E (Exclusive)** state: the line is clean and no other cache holds it. A write from E proceeds silently to M with no bus transaction at all. The additional requirement is a shared signal on the interconnect by which a requesting core learns whether any other cache responded.

MOESI adds **O (Owned)**: a line that is dirty and shared, with one cache designated owner and responsible for supplying it to requesters. This permits sharing of modified data without an intervening writeback to memory. AMD processors use MOESI; Intel uses a MESI derivative with an additional forwarding state. Full transition tables for all three protocols are deferred to Appendix F.

3.3 Snooping and Directories

The protocols above describe states and transitions; a separate question is how a core's requests reach the others.

Snooping broadcasts every request on a shared medium that all caches observe. It is simple and provides a natural total order on transactions — the order in which they win the bus — but requires bandwidth proportional to the number of cores, and does not scale past roughly sixteen.

Directory protocols maintain, for each line, a record of which caches hold copies, and send point-to-point messages only to those. This scales, at the cost of an extra indirection through the directory (a three-hop rather than two-hop miss) and of storage for the sharer vectors. All modern large multicores are directory-based. The prescribed text treats directories in §12.4.6; the interconnect topologies over which the messages travel are the subject of §12.6 and are relevant to this paper only insofar as they determine coherence latency, a parameter of the simulator in Chapter 8.

3.4 Coherence versus Consistency

We can now state the distinction with the precision the rest of the paper requires.

Table 3.1: The division of labour between coherence and consistency.

	Coherence	**Consistency**
Scope	One address at a time	Across different addresses
Guarantee	A single total order of writes per location, visible to all cores	Which program-order relations between operations are preserved globally
Mechanism	MSI / MESI / MOESI, snooping or directory	Store buffer discipline, fences, speculation
Where it acts	At the cache	Between the core and the cache
Failure mode	A core reads a stale value of a location	Two cores disagree about the relative order of accesses to different locations

The critical structural fact is the fourth row. *The store buffer sits between the core and the cache*, and therefore in front of the coherence protocol entirely. A store resting in the buffer has not yet been presented to the coherence mechanism; there is nothing for the protocol to order, invalidate or broadcast. Coherence cannot police an operation it has not seen.

This is why the outcome of §1.1 survives a perfect coherence implementation, and it is why the two topics — which are frequently taught together and are treated in adjacent subsections of the prescribed text — must be kept firmly apart.

Example 3.1 (Coherence holds; the anomaly persists)**.** In Listing 1.1, consider the accesses to `x`. There are two: Core 0's store and Core 1's load. Coherence requires only that they be totally ordered and that the load return the value written by the most recent preceding store in that order. Placing the load *before* the store in coherence order satisfies this requirement exactly, and yields `r1` $= 0$. The symmetric argument for `y` yields `r0` $= 0$. Both coherence invariants hold throughout.

3.5 What Coherence Does Contribute

It would be wrong to conclude that coherence is irrelevant to consistency. It contributes two things that Chapter 4 will use.

First, it supplies the *coherence order* relation $\xrightarrow{\text{co}}$, a total order on the stores to each location, which is one of the primitive relations from which consistency models are constructed.

Second, and more practically, the invalidation messages it generates constitute a ready-made notification service. When a core speculatively executes a load early, the arrival of an invalidation for that line is precisely the signal that the speculation may

have been observable. An optimistic implementation of sequential consistency needs a detection mechanism, and coherence has already built one for other reasons. This observation, developed in Chapter 9, is what makes speculative SC comparatively cheap.

3.6 Summary

Coherence is a solved problem, correctly implemented in shipping hardware, and it guarantees per-location sequential consistency. It says nothing about the ordering of accesses to distinct locations, and cannot, because the mechanism that reorders them operates upstream of it. The specification of that cross-location ordering is a separate architectural decision, and it is to the space of such decisions that we now turn.

Chapter 4

The Design Space of Memory Consistency Models

This chapter is the formal core of the paper. It introduces a single vocabulary of ordering relations, expresses five consistency models as successively weaker constraints within it, and establishes the containment hierarchy between them. The formalism follows the execution-witness and access-graph treatment of Sarangi [14, Ch. 9], with terminology reconciled to that of Nagarajan et al. [13] where the two differ.

4.1 Executions and Relations

4.1.1 Memory Events

An *execution* of a multithreaded program is a set E of memory events. Each event $e \in E$ is a load $\mathrm{L}(a)$ or a store $\mathrm{S}(a)$ to some address a, issued by an identified core, and carrying a value. Non-memory instructions are irrelevant to the model and are omitted.

4.1.2 The Four Primitive Relations

Program order, $\xrightarrow{\mathbf{po}}$. A per-core total order. $e_1 \xrightarrow{\mathrm{po}} e_2$ if both are issued by the same core and e_1 precedes e_2 in that core's instruction stream. This relation is given by the program and is not subject to negotiation.

Reads-from, $\xrightarrow{\mathbf{rf}}$. $\mathrm{S}(a) \xrightarrow{\mathrm{rf}} \mathrm{L}(a)$ if the load returns the value written by that store. Every load reads from exactly one store (taking the initialisation of memory as a store), so $\xrightarrow{\mathrm{rf}}$ is a function on loads.

Coherence order, $\xrightarrow{\mathbf{co}}$. For each address a, a total order on all stores to a. This is exactly the order guaranteed by the coherence protocol of Chapter 3, and its existence is the formal content of the SWMR invariant.

From-read, $\xrightarrow{\mathbf{fr}}$. A derived relation. $\mathrm{L}(a) \xrightarrow{\mathrm{fr}} \mathrm{S}(a)$ if the load reads from some store that precedes this store in $\xrightarrow{\mathrm{co}}$; informally, the load happened before this store overwrote the value it observed. Formally,

$$\xrightarrow{\mathrm{fr}} = \xrightarrow{\mathrm{rf}}^{-1} ; \xrightarrow{\mathrm{co}}$$

where ; denotes relational composition.

4.1.3 The Central Definition

Every model in this chapter has the same shape. Each defines a subset of program order, called *preserved program order* and written $\xrightarrow{\text{ppo}}$, consisting of those program-order pairs the model promises to maintain globally. An execution is permitted by the model exactly when the combined relation is acyclic.

Definition 4.1 (Model-permitted execution)**.** Let M be a consistency model with preserved program order $\xrightarrow{\text{ppo}}_M$. An execution E is permitted by M if and only if the relation

$$\xrightarrow{\text{ppo}}_M \cup \xrightarrow{\text{rf}} \cup \xrightarrow{\text{co}} \cup \xrightarrow{\text{fr}}$$

is acyclic over E.

A cycle in this relation is a proof that no total order of events consistent with the model can produce the execution. The models therefore differ in exactly one respect — how much of $\xrightarrow{\text{po}}$ survives into $\xrightarrow{\text{ppo}}$ — and the entire design space can be presented as a single table. That table is table 4.1.

4.2 Litmus Tests

A *litmus test* is a minimal multithreaded program, together with a distinguished final state, used to distinguish models. If a model permits the state, the hardware may produce it; if not, it may not.

4.2.1 SB: Store Buffering

Listing 1.1 of Chapter 1, in litmus form:

Core 0	**Core 1**
`St(x,1)`	`St(y,1)`
`Ld(y)` $\rightarrow$ `r0`	`Ld(x)` $\rightarrow$ `r1`

Target: $\texttt{r0} = 0 \wedge \texttt{r1} = 0$

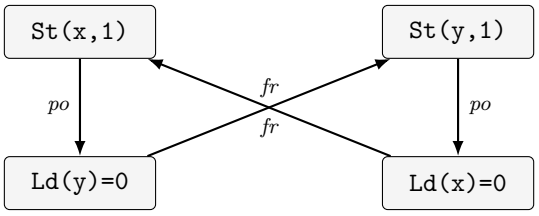

Figure 4.1: Access graph for the SB litmus test with the target outcome. The four edges form a cycle, so the outcome is forbidden by any model that preserves store $\rightarrow$ load program order. TSO does not preserve it, and the cycle is therefore not a cycle in $\xrightarrow{\text{ppo}}_{\text{TSO}} \cup \xrightarrow{\text{rf}} \cup \xrightarrow{\text{co}} \cup \xrightarrow{\text{fr}}$.

Each load reads the initial value, so each load is $\xrightarrow{\text{fr}}$-before the other core's store. Together with the two $\xrightarrow{\text{po}}$ edges this closes a cycle. Under SC, where $\xrightarrow{\text{ppo}} = \xrightarrow{\text{po}}$, the cycle is present and the outcome is forbidden. Under TSO, where store $\rightarrow$ load pairs are dropped from $\xrightarrow{\text{ppo}}$, both $\xrightarrow{\text{po}}$ edges vanish and no cycle remains: the outcome is permitted.

4.2.2 MP: Message Passing

Core 0	Core 1
`St(data,1)`	`Ld(flag)` $\rightarrow$ `r0`
`St(flag,1)`	`Ld(data)` $\rightarrow$ `r1`

Target: $\texttt{r0} = 1 \wedge \texttt{r1} = 0$

This is the idiom underlying every publication protocol: write the payload, then set a flag; on the reader, test the flag, then read the payload. The target outcome — flag observed set, payload observed stale — requires either that Core 0's two stores were reordered (a store $\rightarrow$ store relaxation) or that Core 1's two loads were reordered (a load $\rightarrow$ load relaxation). TSO permits neither, and MP is accordingly forbidden on x86. Weak models permit both, and MP is observable on ARM.

4.2.3 LB, IRIW, WRC and the Coherence Tests

Four further tests complete the discriminating set used in this paper.

LB (Load Buffering). Each core loads one variable and stores the other; the target has each load returning the other core's store. Requires load $\rightarrow$ store reordering. Forbidden under TSO, permitted under ARM and POWER in principle, though rarely observed.

IRIW (Independent Reads of Independent Writes). Two writer cores and two reader cores; the target has the readers disagreeing about the order of the two writes. Distinguishes *multicopy atomic* models, in which a store becomes visible to all cores simultaneously, from non-multicopy-atomic ones. POWER and ARMv7 permit it; ARMv8, revised in 2018, does not.

WRC (Write-to-Read Causality). Three cores in a causal chain; tests whether causality is transitive.

CoRR, CoWR, CoRW. Single-address tests that check coherence itself rather than consistency. Every model forbids their target outcomes; they serve as controls, and any hardware or simulator producing them is defective.

4.3 The Models

4.3.1 Sequential Consistency

Definition 4.2 (Sequential consistency, after Lamport [10])**.** A multiprocessor is sequentially consistent if the result of any execution is the same as if the operations of all processors were executed in some sequential order, and the operations of each individual processor appear in that sequence in the order specified by its program.

In the framework of definition 4.1, $\xrightarrow{\text{ppo}}_{\text{SC}} = \xrightarrow{\text{po}}$: all four program-order pairs are preserved. SC is the model that validates the informal reasoning of §1.1, and it is the only model in this chapter that does.

4.3.2 Total Store Order

TSO relaxes exactly one pair: a store may be reordered with a *later* load to a different address. This corresponds precisely to a FIFO store buffer with store-to-load forwarding, and nothing else. It is the model of SPARC TSO and, with minor differences of specification, of x86.

Two further properties characterise TSO. The store buffer is FIFO, so store $\rightarrow$ store order is preserved. And TSO is *multicopy atomic*: a store that leaves a core's buffer becomes visible to all other cores at once, which is why IRIW is forbidden.

4.3.3 Partial Store Order

PSO additionally relaxes store $\rightarrow$ store, modelling a non-FIFO store buffer that may drain entries out of order or coalesce them. It was defined by SPARC and is of limited practical importance today, but it is a necessary intermediate point in the hierarchy and it isolates the store $\rightarrow$ store relaxation for study.

4.3.4 Weak Ordering and RMO

Weak ordering, introduced by Dubois, Scheurich and Briggs [7], abandons the attempt to preserve any program order between ordinary accesses. It instead divides accesses into *data* and *synchronisation* operations and guarantees ordering only at synchronisation points. SPARC RMO, Alpha, POWER, ARMv7 and RISC-V's RVWMO are all of this family, differing in detail — notably in whether they are multicopy atomic and in which dependencies are respected.

Three dependencies survive in essentially every weak model, because violating them would break single-threaded semantics:

Address dependency. A load whose result computes the address of a later access.

Data dependency. A store whose value is computed from an earlier load.

Control dependency. A branch conditional on a load. Notably, control dependencies order later *stores* but not later *loads*, since the latter may be executed speculatively past the branch — a subtlety that has caused documented bugs in production code.

4.3.5 Release Consistency

Release consistency, due to Gharachorloo et al. [8], refines weak ordering by distinguishing two kinds of synchronisation:

- An *acquire* orders itself before all subsequent accesses.
- A *release* orders all preceding accesses before itself.

Neither constrains operations in the other direction, so the fences are strictly cheaper than a full barrier. This is the model exposed by C++11's `memory_order_acquire` and `memory_order_release` and implemented directly in ARMv8's `LDAR` and `STLR` instructions.

4.3.6 Summary Table

Table 4.1: Program-order pairs preserved by each model. A tick indicates the pair is in $\xrightarrow{\text{ppo}}$; a cross indicates it may be reordered.

Model	S→L	S→S	L→L	L→S	MCA	Deployed in
SC	✓	✓	✓	✓	✓	—
TSO	×	✓	✓	✓	✓	x86, SPARC
PSO	×	×	✓	✓	✓	SPARC
Weak/RMO	×	×	×	×	varies	ARM, POWER, RISC-V
RC	×	×	×	×	varies	C++11, ARMv8

MCA denotes multicopy atomicity. ARMv8 became multicopy atomic in its 2018 revision; POWER and ARMv7 are not.

4.4 The Containment Hierarchy

Let $\mathcal{X}(M)$ denote the set of executions permitted by model M.

Theorem 4.1 (Containment)**.** $\mathcal{X}(\text{SC}) \subsetneq \mathcal{X}(\text{TSO}) \subsetneq \mathcal{X}(\text{PSO}) \subsetneq \mathcal{X}(\text{Weak})$.

Proof sketch. Containment. By construction $\xrightarrow{\text{ppo}}_{\text{Weak}} \subseteq \xrightarrow{\text{ppo}}_{\text{PSO}} \subseteq \xrightarrow{\text{ppo}}_{\text{TSO}} \subseteq \xrightarrow{\text{ppo}}_{\text{SC}}$, since each model is obtained from its predecessor by deleting pairs. If a relation is acyclic then any subrelation of it is acyclic; hence a superset of $\xrightarrow{\text{ppo}}$ generates a superset of the cycles, and so a subset of the permitted executions.

Strictness. Each inclusion is witnessed by a litmus test permitted by the weaker model and forbidden by the stronger: SB separates SC from TSO; MP with the stores reordered separates TSO from PSO; MP with the loads reordered separates PSO from Weak. Figure 4.1 gives the argument in the first case; the others are structurally identical and are given in full in Appendix B. □

The hierarchy has a practical reading. A program correct under a weak model is correct under every stronger one, so code written for ARM runs correctly on x86. The converse fails, and this asymmetry is the source of a well-known class of porting defect.

4.5 Fences

A fence restores ordering that the model does not provide by default. Its semantics are stated as an addition to $\xrightarrow{\text{ppo}}$: a full fence F contributes $\{(e_1, e_2) : e_1 \xrightarrow{\text{po}} F \xrightarrow{\text{po}} e_2\}$.

Table 4.2: Ordering instructions in deployed instruction sets.

ISA	Instruction	Effect
x86	`MFENCE`	Full barrier; drains the store buffer.
x86	`LOCK` prefix	Atomic RMW; carries full-barrier semantics.
ARMv8	`DMB ISH`	Full data memory barrier, inner shareable.
ARMv8	`LDAR` / `STLR`	Load-acquire, store-release.
POWER	`sync` / `lwsync`	Heavyweight and lightweight sync.
RISC-V	`FENCE` *pred,succ*	Bit-encoded predecessor/successor sets.

The RISC-V encoding is worth noting: rather than offering a fixed menu, it exposes four predecessor and four successor bits, so that the programmer requests exactly the orderings needed. This is the most explicit acknowledgement in any deployed ISA that fence strength is a continuum rather than a binary.

Fences are not free. Restoring an ordering means reintroducing the stall the relaxation was introduced to avoid; a full barrier on x86 costs on the order of 50–100 cycles. Quantifying this cost on real hardware is one of the measurements specified in Chapter 6.

4.6 Data-Race-Free Programming

The models above are hardware contracts. Programmers work in languages, and the bridge between the two is the data-race-freedom theorem of Adve and Hill [2].

Definition 4.3 (Data race)**.** Two accesses conflict if they target the same address and at least one is a store. An execution has a data race if two conflicting accesses from different cores are unordered by synchronisation. A program is data-race-free (DRF) if no sequentially consistent execution of it contains a race.

Theorem 4.2 (DRF, informally)**.** *A DRF program executing on a weakly ordered machine that respects synchronisation operations produces only sequentially consistent results.*

This is the settlement on which modern practice rests, and it is the intellectual foundation of both the Java Memory Model [11] and the C++11 model [5]. Its bargain is explicit: *annotate your synchronisation correctly and you may reason sequentially.* Its weakness is equally explicit — the guarantee is void for racy programs, and racy programs are common. Chapter 11 returns to whether this bargain was a good one.

4.7 Summary

Five models, one formalism, one table. The models differ only in how much of program order they preserve, and they form a strict hierarchy. What the table does not show is cost: it says what each model forbids, not what forbidding it is worth. Chapter 5 recounts how the field came to settle where it did, and Chapter 6 sets out how this paper proposes to measure whether that settlement was correct.

Chapter 5

Historical Evolution

The models of Chapter 4 are usually presented as a design space, as though an architect had surveyed the options and chosen. The history is less tidy. Each model was a response to a specific difficulty, several were formalised only after hardware implementing them had already shipped, and at least two widely deployed specifications were subsequently found not to describe the machines they governed. This chapter traces that history, because the argument this paper examines — that sequential consistency is too expensive — was made at a particular moment under particular assumptions, and those assumptions are visible only in context.

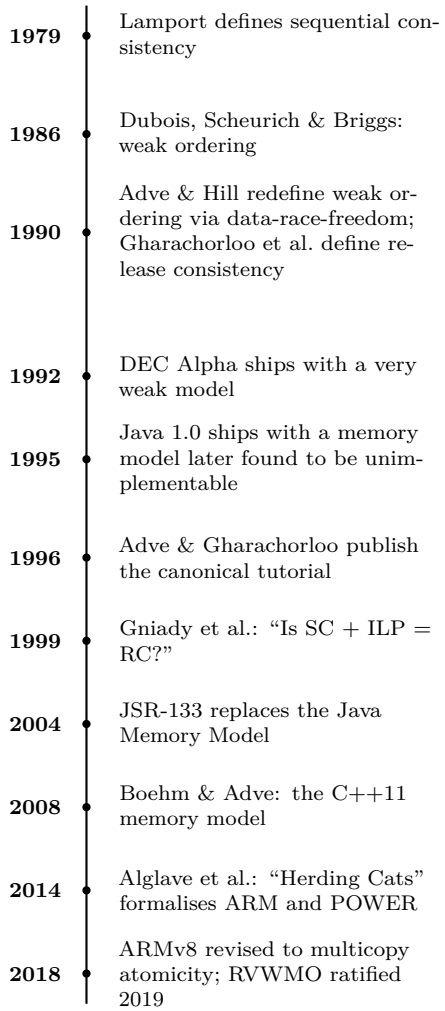

Figure 5.1: Chronology of memory consistency models.

5.1 1979: The Definition

Lamport's paper [10] is three pages long and does not argue that sequential consistency is desirable. It assumes so, and asks what a hardware implementation must satisfy to achieve it. The two requirements it gives — each processor issues its requests in program order, and requests to a single memory module are serviced in arrival order — were, on the machines of 1979, neither surprising nor expensive. The paper is best read as a formalisation of what everyone already believed the machine did.

The significance of the paper is that it made the assumption explicit and therefore falsifiable. Once SC had a definition, it became possible to build a machine that demonstrably did not satisfy it, and to argue that doing so was a good idea.

5.2 1986–1990: The Case for Relaxation

Dubois, Scheurich and Briggs [7] made that argument. Their observation was that SC's requirements bind at every memory operation, whereas programs actually require ordering only at the few points where threads communicate. Enforcing ordering everywhere to protect the rare case is a poor trade. *Weak ordering* follows: distinguish synchronisation accesses from data accesses, and guarantee ordering only around the former.

Adve and Hill [2] reformulated this from the programmer's side. Rather than defining weak ordering as a hardware behaviour, they defined it as a *contract*: if the programmer writes a data-race-free program, the hardware will appear sequentially consistent. This inversion is the single most consequential idea in the field. It made weak hardware defensible — the programmer is not exposed to reordering, provided the program is properly synchronised — and it supplied the framework in which every subsequent language-level memory model has been written.

Gharachorloo et al. [8] refined it further with release consistency, observing that acquire and release semantics need constrain only one direction each, and so admit cheaper implementations than full barriers.

By 1990 the intellectual case was complete: SC is expensive, relaxation is safe for well-written programs, and here is the framework proving it. Hardware followed immediately. The DEC Alpha, designed in this period, adopted one of the weakest models ever shipped.

5.3 The Cost of Being Right

The DRF settlement rested on a premise that received less scrutiny than it deserved: that programmers would in fact write data-race-free programs. The subsequent three decades tested that premise.

Java, 1995 and 2004. Java shipped in 1995 with a memory model intended to give strong guarantees even to racy programs, for security reasons — a sandboxed language cannot let a race corrupt the virtual machine's own invariants. The model was found to be simultaneously too weak to support the idioms programmers used and too strong to permit standard compiler optimisations. It was replaced wholesale by JSR-133 [11] after a multi-year effort. That a memory model for a single language required nine years and a formal committee is itself evidence about the difficulty of the subject.

C++, 2008. C++ had no memory model at all until C++11. Multithreaded C++ was written against the pthreads library, whose specification described synchronisation but not the underlying memory semantics, leaving the correctness of essentially all concurrent C++ formally undefined. Boehm and Adve [5] supplied the missing foundation.

Specifications that did not describe the hardware. Alglave, Maranget and Tautschnig [3] formalised the published ARM and POWER architecture manuals and tested the resulting models against real processors. The models did not match: they permitted executions no chip produced and forbade some that chips did. The response, over the following years, included ARM's 2018 revision of ARMv8 to multicopy atomicity — a *strengthening* of a shipped architecture, undertaken because the weaker model was too difficult to reason about.

5.4 1999–2011: The Counter-Argument

While the industry moved right along the spectrum of Figure 1.1, a line of work asked whether it had needed to.

Gniady, Falsafi and Vijaykumar [9] posed the question in their title: *Is SC + ILP = RC?* Their observation was that the 1980s cost estimate for SC assumed the core would stall. On a core that speculates, it need not: the load may execute early and be squashed if a coherence invalidation proves the speculation observable. They reported that SC so implemented came close to release consistency in performance.

Subsequent work extended the mechanism. BulkSC [6] abandoned per-instruction reasoning in favour of executing large chunks of instructions atomically, so that consistency is guaranteed at chunk boundaries — the same insight that underlies transactional memory. InvisiFence [4] reduced the cost of the fences themselves.

Marino et al. [12] closed the remaining gap by addressing the compiler, showing that a compiler forbidden from reordering shared memory accesses loses only a few per cent of performance — far less than had been assumed.

5.5 The Unresolved Position

The situation as it stands is peculiar. The academic literature contains a substantial body of evidence that sequential consistency can be implemented at modest cost on a speculative core. The commercial landscape contains no processor that does so. Meanwhile, ARM has strengthened its model, RISC-V's RVWMO was ratified only after extended debate about how weak to make it, and the difficulty of programming weak models is a matter of documented record rather than speculation.

At least four explanations are available, and they are not mutually exclusive.

1. **The measurements do not scale.** The speculative-SC results were obtained at modest core counts. Conflict frequency, and therefore squash rate, grows with cores.

2. **Energy, not time.** A squash discards completed work. In a power-constrained design — which every mobile design is — this may be decisive even where execution time is not.

3. **Verification.** Verifying a speculative implementation against a strong specification is substantially harder than verifying a straightforward implementation against a weak one.

4. **Architectural commitment.** A memory model is a permanent contract. Weakening one later is impossible; strengthening one, as ARM did in 2018, is possible but rare. The safe choice for a vendor is to promise little.

Distinguishing among these is beyond the scope of a single term paper, but the first can be tested directly, and doing so is the purpose of the simulation study described in the next chapter.

Chapter 6

Methodology

This chapter specifies the experiments whose results appear in later versions of this paper. It is stated in advance and in full so that the design may be evaluated independently of the outcome.

Three instruments are used, answering three different kinds of question. Litmus testing on real hardware establishes what commercially available machines actually do. A configurable simulator establishes what each memory model costs, holding everything else fixed. An axiomatic checker establishes that the simulator implements the models it claims to implement.

6.1 Instrument 1: Litmus Testing on Hardware

6.1.1 Test Harness

Each litmus test is realised as a C program with one pinned thread per core, executing the test body in a loop of $N = 10^7$ iterations. Each iteration resets the shared variables, releases the threads, executes the body, and records the observed final state. The design constraints are as follows.

Thread placement. Threads are pinned with `pthread_setaffinity_np`. Placement is a controlled variable, not an incidental one: two threads on sibling SMT contexts of the same physical core share a store buffer and will produce different results from two threads on distinct cores. The topology reported by `lscpu -e` is recorded and reported with every result.

Synchronisation between iterations. A conventional `pthread_barrier` is unsuitable: barriers contain full fences, which suppress precisely the window under measurement. A lightweight sense-reversing spin barrier built from relaxed atomics is used instead, and its own fence content is documented.

Preventing compiler interference. Shared variables are declared `_Atomic` and accessed with `memory_order_relaxed`, which forbids the compiler from eliminating or reordering the accesses while emitting no hardware fences. Using `volatile` instead is common practice but is not a substitute: `volatile` constrains the compiler only with respect to other volatile accesses. Both variants are compiled and compared, and the difference is itself reported.

Variable placement. Each shared variable is placed on its own cache line by padding to 64 bytes. Without this, false sharing changes the coherence behaviour and therefore the observation rate.

6.1.2 Platforms

Table 6.1: Target platforms. Exact models to be recorded in the results chapter.

Class	Model	Expected	Role
x86-64	TSO	SB observable; MP, LB not	strong-model baseline
AArch64	ARMv8	SB, MP, LB observable; IRIW not	weak-model comparison

The AArch64 platform may be a single-board computer, an Android device under Termux, or an institutional server; whichever is used, its microarchitecture and core topology are recorded. The IRIW test requires at least four cores and is reported only where the platform supports it.

6.1.3 Measurements Reported

1. **Observation frequency** per test per platform, over 10^7 iterations, with ten repetitions reported as mean and standard deviation. Reporting frequency rather than a binary observed/not-observed classification is a deliberate choice: the gap between what an architecture permits and what a particular implementation exhibits is one of the paper's subjects, and it is invisible in a binary encoding.

2. **The effect of fences.** Each test is re-run with the appropriate fence inserted. The expected result is that every observation frequency falls to zero; any test for which it does not indicates either a misplaced fence or a misunderstanding of the fence's semantics, and would require investigation.

3. **Fence cost in cycles**, measured by timing a tight loop with and without the barrier instruction using `rdtsc` on x86 and the cycle counter on AArch64, with frequency scaling disabled.

4. **Sensitivity** to thread placement and to inserted delay between the store and the load, establishing the width of the vulnerability window.

6.1.4 Threats to Validity

Litmus testing establishes that a behaviour *can* occur; it can never establish that one cannot. A zero observation count is consistent with the behaviour being architecturally forbidden, with it being permitted but not exhibited by this implementation, and with the harness suppressing it. This paper therefore reports observations as lower bounds and pairs every measurement with the corresponding architectural statement from the vendor manual. Where the two disagree — permitted but never observed — the discrepancy is reported as a finding rather than resolved by assumption.

6.2 Instrument 2: The Simulator

6.2.1 Rationale

Hardware answers what x86 and ARM do. It cannot answer what SC would cost, because no SC hardware exists to measure. That question requires a machine in which

the memory model is a parameter.

6.2.2 Architecture

The simulator models N cores, each with a private L1, connected by a directory protocol over an interconnect of configurable latency. Its distinguishing feature is that the ordering policy is an interchangeable component: a single interface decides, for each memory operation, whether it may issue given the operations still outstanding. Four implementations of that interface are provided — SC-naive, SC-speculative, TSO and Weak — and *no other part of the simulator differs between configurations*. The memory model is thereby the only independent variable.

Table 6.2: Simulator configuration parameters and their default values.

Parameter	**Default**	**Swept over**
Cores	8	2, 4, 8, 16, 32
Store buffer entries	32	8, 16, 32, 64
Load queue entries	32	16, 32, 64
L1 size / assoc.	32 kB / 8	fixed
Coherence	MESI directory	fixed
Interconnect latency	20 cycles	10, 20, 40
Memory latency	200 cycles	fixed
Memory model	—	SC-naive, SC-spec, TSO, Weak

6.2.3 Speculative SC

The SC-speculative policy is the one whose viability the paper tests. Its operation is as follows.

1. A load is permitted to issue even where SC would require it to wait, and its address is retained in the load queue marked speculative.
2. Incoming coherence invalidations are matched against speculative load queue entries.
3. A match indicates that a remote store to a speculatively read line occurred within the window, so the speculation was potentially observable. The load and all younger instructions are squashed and re-executed.
4. Absent a match, the load retires and the relaxation is unobservable.

Both required mechanisms already exist in the modelled core: coherence invalidations are generated by the protocol regardless, and the squash path is the branch-misprediction recovery path. The added hardware is an address comparator against the load queue.

6.2.4 Workloads

Standard benchmark suites are unsuitable as the primary workload. SPEC and PARSEC are dominated by cache and branch behaviour; the memory model contributes little and the effect under study would be buried in noise. Workloads are therefore selected for communication density.

Table 6.3: Simulated workloads.

Tier	Workload	Stresses
Sync	Spinlock counter	Fence cost on acquire and release under contention
Sync	Dekker mutual exclusion	Store → load ordering directly
Sync	SPSC ring buffer	Acquire/release pairs in steady state
Sync	Treiber stack	Atomic RMW and ordering around it
Sync	Sense-reversing barrier	All-core synchronisation
Kernel	Parallel reduction	Mostly-private access with periodic combining
Kernel	Shared-frontier BFS	Irregular sharing
Kernel	Atomic histogram	Contended atomic updates

The two tiers serve different purposes. The synchronisation microbenchmarks isolate the effect; the kernels test whether it survives into code that performs useful work. It is anticipated that the gap between models narrows substantially in the second tier, and that outcome, if observed, is a result and not a disappointment.

6.2.5 Metrics

Per run, the simulator emits: total cycles; IPC; stall cycles decomposed by cause (store buffer full, fence drain, cache miss, coherence wait); squash count; coherence messages; mean store buffer occupancy.

The headline metric is execution time normalised to Weak. The explanatory metric is the decomposition of stall cycles: the hypothesis under test predicts that SC-naive's ordering stalls are largely *converted* into squash cycles under SC-speculative, and that squash cycles are fewer because they are incurred only on genuine conflicts. A result without this decomposition would establish a correlation without a mechanism.

6.2.6 Experimental Protocol

Each configuration is run once as warm-up and discarded, then ten times with distinct seeds. Results are reported as mean and standard deviation with error bars on all figures. The full sweep is 8 workloads × 4 models × 5 core counts × 10 repetitions, or 1600 runs, driven by a single script and emitting one CSV row per run. All figures are generated from those CSVs by scripts under version control, so that the figures are reproducible from the data by one command.

A methodological commitment. Every number in the results chapters originates from this simulator under the configuration recorded alongside it. No comparison is made against published figures obtained on other simulators or other hardware, since differing microarchitectural assumptions would render such comparisons meaningless. The value of the instrument lies entirely in its being a controlled experiment.

6.3 Instrument 3: Verification

A performance comparison between memory models is worthless if the implementations do not in fact implement those models. The third instrument establishes that they do.

For each litmus test and each model, two sets of final states are computed:

1. The *permitted* set, by enumerating all candidate executions and applying the acyclicity criterion of definition 4.1 directly.

2. The *produced* set, by exhaustively enumerating all interleavings the simulator admits for that test under that ordering policy.

The two sets must be equal. A produced state outside the permitted set is a bug in the ordering policy. A permitted state never produced indicates that the implementation is stricter than the model requires — not a correctness error, but a fact that must be reported, since it would bias the performance comparison in that model's favour.

Litmus tests are small enough that exhaustive enumeration is tractable, and the checker is a distinct piece of software from the timing simulator, so that a common bug is unlikely to mask itself in both.

6.4 Tooling and Reproducibility

The simulator is written in C++ and emits CSV. Figures are produced in Python with pandas and matplotlib, one script per figure, exported as vector PDF. Litmus tests are C11 with pthreads. The paper is written in LaTeX with all figures in TikZ, and references are maintained in Zotero and exported to BibTeX. Random seeds are fixed, a single `run_all.sh` reproduces every result, and the repository is cited in the final version.

6.5 Anticipated Results and Falsifiability

The hypothesis under test is that speculative sequential consistency is substantially cheaper than naive sequential consistency, and close to weak ordering, on a core that already speculates. Concretely, the paper predicts:

- SC-naive between 1.5× and 2× the execution time of Weak on the synchronisation tier;
- SC-speculative within 1.05× to 1.15× of Weak on the same tier;
- the advantage of SC-speculative narrowing as core count rises, driven by squash rate.

Stating these in advance makes the study falsifiable. Should SC-speculative prove expensive at all core counts, or should its cost prove insensitive to squash rate, the hypothesis is refuted and the historical verdict is vindicated — which would be a result of equal interest and is reported as such.

Chapter 7

Source Texts and Their Coverage

The course requires that any textbook consulted beyond the prescribed one be identified in full, and that the portion of its table of contents relied upon be reproduced so that the correspondence with this paper is verifiable. This chapter does that. It also states, for each source, which chapter of this paper draws on it and for what.

Sources are grouped into three tiers. Tier 1 texts are read in full and carry the technical weight of the paper. Tier 2 texts are consulted for specific treatments and cross-checks. Tier 3 comprises the vendor architecture manuals, which are the authoritative statement of what each shipping machine actually promises and are therefore the primary source wherever this paper makes a claim about a particular instruction set.

7.1 Tier 1: Primary Texts

7.1.1 Sarangi, *Basic Computer Architecture* (prescribed)

Author	Smruti R. Sarangi
Title	Basic Computer Architecture
Version	3.09 (8 October 2025)
Publisher	White Falcon Publishing
Licence	CC BY-ND 4.0; PDF freely available
URL	`https://srsarangi.github.io/archbook/archbook.pdf`

Relevant portion of the table of contents:

§	Title	Page
Chapter 10 — Principles of Pipelining		
10.4	Pipeline Hazards	420
10.9	Performance Metrics	463
10.11.4	Out-of-Order Pipelines	492
Chapter 11 — The Memory System		
11.1	Overview (locality, hierarchy, cache blocks)	506
11.2	Caches	517
11.3	The Memory System	532
Chapter 12 — Multiprocessor Systems		
12.2.2	Shared Memory vs Message Passing	571
12.2.3	Amdahl's Law	576
12.3	Design Space of Multiprocessors	578
12.4	MIMD Multiprocessors	579
12.4.1	Logical Point of View	579
12.4.2	Coherence	581
12.4.3	**Memory Consistency**	583
12.4.4	Physical View of Memory	593
12.4.5	Shared Caches	596
12.4.6	Coherent Private Caches	596
12.4.7	Implementing a Memory Consistency Model*	603
12.4.8	Multithreaded Processors	608
12.6	Interconnection Networks	620
Appendix A — Case Studies of Real Processors		
A.1.2	ARM Cortex-A8	734
A.1.3	ARM Cortex-A15	736
A.3.2	Intel Sandy Bridge	746

Table 7.1: Sections of the prescribed text spanned by this paper. Sections marked $*$ are designated advanced in the text.

Use in this paper. §12.4.2 and §12.4.3 are the sections this paper extends; they supply the coherence/consistency distinction developed in Chapter 3 and the sequential-consistency treatment developed in Chapter 4. §12.4.7 is the direct antecedent of the implementation discussion. §11.2 and §10.11.4 supply the microarchitectural background of Chapter 2, §10.9 the performance-metric framework used in Chapter 6, and §12.6 the interconnect parameters of the simulator.

7.1.2 Sarangi, *Next-Gen Computer Architecture*

Author	Smruti R. Sarangi
Title	Next-Gen Computer Architecture: Till the End of Silicon
Publisher	White Falcon Publishing, October 2023
ISBN	8119510143
Note	First edition published by McGraw-Hill (2021) as *Advanced Computer Architecture*, ISBN 97893907
Licence	CC BY-ND 4.0; PDF freely available
URL	`https://www.cse.iitd.ac.in/~srsarangi/advbook/`

This is the companion volume to the prescribed text, aimed at final-year undergraduates, and it is the single most important source for this paper after the prescribed book. Chapter 9 is devoted entirely to the subject and runs to approximately 118 pages.

§	Title	Used in
Chapter 9 — Multicore Systems: Coherence, Consistency and Transactional Memory		
9.1	Parallel programming and hardware threads	Ch. 2
9.2	Theoretical foundations	Ch. 4
9.3	Sequential consistency, PLSC, and coherence	Ch. 3, Ch. 4
9.4	**Execution witnesses, access graphs, causal graphs**	Ch. 4
9.5	Cache coherence: snoopy and directory protocols	Ch. 3
9.6	Advanced directory protocols and atomic operations	Ch. 3, Ch. 8
9.7	Memory models and data races	Ch. 4, Ch. 6
9.8	Methods to detect races	Ch. 10
9.9	Transactional memory	Ch. 11
Supporting chapters		
4.3	Load store queue	Ch. 2
5.1	Aggressive speculation	Ch. 9
7	Caches	Ch. 2
8	On-chip network	Ch. 8
App. B	Tejas Architectural Simulator	Ch. 8

Table 7.2: Sections of *Next-Gen Computer Architecture* consulted. Page numbers are omitted because they differ between the McGraw-Hill and White Falcon printings and between PDF versions; section numbering is stable.

Use in this paper. §9.4 supplies the formalism of Chapter 4. The execution-witness and access-graph notation used throughout this paper is Sarangi's, chosen in preference to the alternative formulations in the literature so that the notation is continuous with the course. §9.3's treatment of per-location sequential consistency is the source of the PLSC characterisation of coherence in §3.1. §9.7 and §9.8 underpin the data-race material of §4.6 and the verification design of Chapter 10. Appendix B documents the Tejas simulator, evaluated as an alternative to the purpose-built simulator of Chapter 8.

7.1.3 Nagarajan, Sorin, Hill and Wood, *A Primer on Memory Consistency and Cache Coherence*

Authors	Vijay Nagarajan, Daniel J. Sorin, Mark D. Hill, David A. Wood
Title	A Primer on Memory Consistency and Cache Coherence
Edition	Second edition, 2020
Series	Synthesis Lectures on Computer Architecture
Publisher	Springer Nature, Cham
Access	Open access via the OAPEN Library

Ch.	Title	Used in
1	Introduction to Consistency and Coherence	Ch. 1
2	Coherence Basics	Ch. 3
3	Memory Consistency Motivation and Sequential Consistency	Ch. 1, Ch. 4
4	Total Store Order and the x86 Memory Model	Ch. 4, Ch. 7
5	Relaxed Memory Consistency	Ch. 4
6	Coherence Protocols	Ch. 3
7	Snooping Coherence Protocols	Ch. 3
8	Directory Coherence Protocols	Ch. 3, Ch. 8
9	Advanced Topics in Coherence	Ch. 3
10	Consistency and Coherence for Heterogeneous Systems	Ch. 12
11	Specifying and Validating Memory Consistency Models	Ch. 10

Table 7.3: Chapter listing of the Primer, second edition. Page ranges vary between the Springer and OAPEN printings and should be cited from the reader's own copy.

Use in this paper. This is the standard reference on the subject and the source of the SWMR and data-value invariants stated in §3.1. Chapters 3–5 are the direct source for the model definitions of Chapter 4; Chapter 11 informs the verification methodology of Chapter 10. Where its notation and Sarangi's differ, this paper follows Sarangi's and notes the correspondence.

7.2 Tier 2: Supporting Texts

7.2.1 Hennessy and Patterson, *Computer Architecture: A Quantitative Approach*

Sixth edition, Morgan Kaufmann, 2019. Chapter 5, *Thread-Level Parallelism*, is the relevant portion:

§	Title
5.1	Introduction
5.2	Centralized Shared-Memory Architectures
5.3	Performance of Symmetric Shared-Memory Multiprocessors
5.4	Distributed Shared-Memory and Directory-Based Coherence
5.5	Synchronization: The Basics
5.6	**Models of Memory Consistency: An Introduction**
5.7	Cross-Cutting Issues
5.8	Putting It All Together: Multicore Processors and Their Performance

Table 7.4: Hennessy & Patterson, 6th ed., Chapter 5.

Use in this paper. §5.6 is a compact treatment of consistency that is useful as a cross-check but does not go beyond what the Tier 1 sources provide. §5.3 and §5.8 are consulted for the benchmarking methodology of Chapter 6, since the quantitative framework of this text is the standard one for reporting multiprocessor performance.

7.2.2 Culler, Singh and Gupta, *Parallel Computer Architecture*

Morgan Kaufmann, 1998. Dated in its hardware examples, but it contains the most extended textbook treatment of relaxed consistency models available, written while

the debate this paper examines was still live — which makes it a primary source for Chapter 5 as much as a textbook.

Ch.	Title	Used in
5	Shared Memory Multiprocessors (§5.2 coherence, §5.3 consistency)	Ch. 3, Ch. 4
6	Snoop-based Multiprocessor Design	Ch. 3
8	Directory-based Cache Coherence	Ch. 3
9	Hardware-Software Tradeoffs (§9.1 relaxed memory consistency models)	Ch. 4, Ch. 5

Table 7.5: Culler, Singh & Gupta, chapters consulted.

7.2.3 Herlihy, Shavit, Luchangco and Spear, *The Art of Multiprocessor Programming*

Second edition, Morgan Kaufmann, 2020. Consulted for the software side: the correctness conditions for concurrent objects, spin-lock implementations, and the lock-free structures used as simulator workloads in Chapter 6 (Treiber stack, Michael–Scott queue). Appendix B of that text covers the hardware memory model from the programmer's perspective and is a useful counterweight to the architect's framing of the Tier 1 sources.

7.3 Tier 3: Architecture Manuals

Where this paper states what a particular instruction set guarantees, the vendor manual is the authoritative source and is cited in preference to any textbook.

ISA	Document	Relevant portion
x86-64	Intel 64 and IA-32 Architectures Software Developer's Manual, Vol. 3A	Chapter on Memory Ordering; the store-buffer and `MFENCE` specification
AArch64	Arm Architecture Reference Manual for A-profile	Ch. B2, The AArch64 Application Level Memory Model; `DMB`, `LDAR`, `STLR`
RISC-V	The RISC-V Instruction Set Manual, Vol. I: Unprivileged ISA	The RVWMO chapter and the `FENCE` bit encoding
Linux	`Documentation/memory-barriers.txt` and the Linux Kernel Memory Model (`tools/memory-model`)	An operational specification maintained by practitioners; used in Ch. 5 and Ch. 11

Table 7.6: Architecture manuals consulted.

7.4 Research Literature

The papers cited in Chapter 5 are the primary literature of the field and are listed in the bibliography rather than tabulated here. Six are load-bearing for this paper's argument and are read in full rather than consulted: Lamport [10], Dubois et al. [7], Adve and Hill [2], Adve and Gharachorloo's tutorial [1], Gniady et al. [9], and Alglave et al. [3].

7.5 A Note on Notation

Four of the sources above use incompatible notation for the same objects. The conventions adopted in this paper are those of Sarangi's *Next-Gen* §9.4, on the grounds of continuity

with the course. Table 7.7 records the correspondence so that a reader working from a different source can follow.

This paper	**Meaning**	**Primer**	**Alglave et al.**
$\xrightarrow{\text{po}}$	program order	program order	`po`
$\xrightarrow{\text{rf}}$	reads-from	`rf`	`rf`
$\xrightarrow{\text{co}}$	coherence order	memory order (per address)	`co`
$\xrightarrow{\text{fr}}$	from-read	— (derived)	`fr`
$\xrightarrow{\text{ppo}}$	preserved program order	ordering rules	`ppo`
PLSC	per-location sequential consistency	SWMR + data value	`sc-per-location`

Table 7.7: Correspondence of notation across sources.

Bibliography

[1] Sarita V. Adve and Kourosh Gharachorloo. Shared memory consistency models: A tutorial. *IEEE Computer*, 29(12):66–76, 1996.

[2] Sarita V. Adve and Mark D. Hill. Weak ordering — a new definition. In *Proc. 17th International Symposium on Computer Architecture (ISCA)*, pages 2–14, 1990.

[3] Jade Alglave, Luc Maranget, and Michael Tautschnig. Herding cats: Modelling, simulation, testing, and data mining for weak memory. *ACM Transactions on Programming Languages and Systems*, 36(2):7:1–7:74, 2014.

[4] Colin Blundell, Milo M. K. Martin, and Thomas F. Wenisch. InvisiFence: Performance-transparent memory ordering in conventional multiprocessors. In *Proc. 36th International Symposium on Computer Architecture (ISCA)*, pages 233–244, 2009.

[5] Hans-J. Boehm and Sarita V. Adve. Foundations of the C++ concurrency memory model. In *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 68–78, 2008.

[6] Luis Ceze, James Tuck, Pablo Montesinos, and Josep Torrellas. BulkSC: Bulk enforcement of sequential consistency. In *Proc. 34th International Symposium on Computer Architecture (ISCA)*, pages 278–289, 2007.

[7] Michel Dubois, Christoph Scheurich, and Fayé Briggs. Memory access buffering in multiprocessors. In *Proc. 13th International Symposium on Computer Architecture (ISCA)*, pages 434–442, 1986.

[8] Kourosh Gharachorloo, Daniel Lenoski, James Laudon, Phillip Gibbons, Anoop Gupta, and John Hennessy. Memory consistency and event ordering in scalable shared-memory multiprocessors. In *Proc. 17th International Symposium on Computer Architecture (ISCA)*, pages 15–26, 1990.

[9] Chris Gniady, Babak Falsafi, and T. N. Vijaykumar. Is SC + ILP = RC? In *Proc. 26th International Symposium on Computer Architecture (ISCA)*, pages 162–171, 1999.

[10] Leslie Lamport. How to make a multiprocessor computer that correctly executes multiprocess programs. *IEEE Transactions on Computers*, C-28(9):690–691, 1979.

[11] Jeremy Manson, William Pugh, and Sarita V. Adve. The Java memory model. In *Proc. 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 378–391, 2005.

[12] Daniel Marino, Abhayendra Singh, Todd Millstein, Madanlal Musuvathi, and Satish Narayanasamy. A case for an SC-preserving compiler. In *Proc. ACM SIGPLAN*

Conference on Programming Language Design and Implementation (PLDI), pages 199–210, 2011.

[13] Vijay Nagarajan, Daniel J. Sorin, Mark D. Hill, and David A. Wood. *A Primer on Memory Consistency and Cache Coherence*. Synthesis Lectures on Computer Architecture. Springer Nature, 2nd edition, 2020.

[14] Smruti R. Sarangi. *Next-Gen Computer Architecture: Till the End of Silicon*. White Falcon Publishing, 2023. Chapter 9: Multicore Systems — Coherence, Consistency and Transactional Memory.

[15] Smruti R. Sarangi. *Basic Computer Architecture*. White Falcon Publishing, version 3.09 edition, 2025. Available at `https://srsarangi.github.io/archbook/`.